



Number One Travel & Tourism

Full Project Status Report

Platform build: Stages 10 through 16 (Production Integration Audit)

Repository	mashhorfoods/mashhor-demo
Branch	claude/awesome-feynman-zyfjar
Latest commit	45fe35c — Stage 16: production-readiness audit and integration runbook
Report date	18 September 2026
Overall platform status	Technically complete through Stage 15B; Stage 16 PARTIALLY COMPLETE (no fabricated integrations)

This report covers the platform end to end: the public website and booking journey (Stages 10-11), customer accounts and the deployable backend (Stages 12-12.1-12.2), the supervisor attribution system (Stage 13), the Admin/Operations Dashboard (Stages 14-15A-15B), and the audit performed before any production integration was attempted (Stage 16). Every status stated in this report is backed by an automated test result, a code citation, or an explicit absence of evidence — nothing here is asserted without a basis.

Contents

1. Executive summary
2. Architecture overview
3. Stage-by-stage summary (10-13)
4. Stage 14 — Admin Dashboard
5. Stage 15 — Operations Control Layer
6. Stage 15A — Business Rules Register
7. Stage 15B — Apply & Verify Business Rules
8. Stage 16 — Production Integration Audit
9. Production Integration Matrix
10. Test results summary
11. Pending business decisions
12. Known limitations
13. Recommended next steps

1. Executive summary

The Number One Travel & Tourism platform is a fully built, internally consistent travel-booking system: a public marketing and search site, a booking journey for flights (live-search architecture, development inventory) and request-based services (hotels, visas, packages, Umrah, medical, transport), a customer account area, a supervisor attribution and portal system, a deployable Node.js/SQLite backend, and a full Admin/Operations Dashboard with a formal, versioned Business Rules Register. Every stage was verified with an automated regression suite before being considered done; the suite currently stands at **21 test suites, 0 failures**, plus internationalisation and accessibility audits with zero findings.

The one deliberate exception is **production connectivity**. No real payment provider, flight/hotel supplier, SMS/email/WhatsApp provider, or hosting account exists for this project. Stage 16 audited the platform's readiness for these connections, found the architecture already anticipates every one of them (adapter seams, environment refusal rules, a verify-before-publish deploy pipeline), found and documented one genuine security gap to close before a real payment provider is wired in, and produced a runbook for exactly what each connection needs. No credential, inventory, or delivery confirmation was fabricated at any point — every NOT_CONNECTED status in this report is a true statement, not a placeholder.

Overall status by area

Area	Status	Evidence
Public website & booking journey	COMPLETE	tests/home, services, detail, destinations, offers, booking, journey — all green
Customer accounts & backend	COMPLETE	tests/account, integration, backend — all green
Supervisor attribution & portal	COMPLETE	tests/supervisor, supervisor-portal — all green
Admin/Operations Dashboard	COMPLETE	tests/ops-portal — 900/900 checks
Business Rules Register	COMPLETE (as a system) — 9/10 rules PENDING by design	tests/backend.mjs Stage 15A/15B sections
Production integration	PARTIALLY COMPLETE	no real provider available; see §8-9

2. Architecture overview

The system is a static frontend (vanilla JS, no framework, Arabic-first with full English support and RTL/LTR layout) talking to a small deployable Node.js backend over a documented HTTP contract (`docs/INTEGRATION.md`). Three completely separate session/credential systems exist side by side — customer, supervisor, and staff (Admin/Operations) — each with its own cookie pair, CSRF token, and database table, verified never to cross over.

Layering

- **UI** — page-specific JS modules, a shared component library, a runtime i18n layer (Arabic/English).
- **Adapters** — every external dependency (auth, booking suppliers, payment, documents, notifications) is behind a registered adapter; a development stand-in and a real implementation share one interface, so swapping one for the other never touches UI code.
- **Backend domain functions** — one file per bounded concern (identity, supervisor, staff, business-rules, routes) — the single place business-critical decisions are made.
- **Persistence** — SQLite via a small query helper, with a linear, tracked migration history (`backend/migrations/001` through `004`).

Roles & isolation

Role	Session cookie	Reachable routes	Notes
Customer	no_session / no_csrf	/auth/*, /me/*	Never sees staff or supervisor data
Supervisor	no_supervisor_session / no_supervisor_csrf	/supervisor/*	Scoped strictly to their own attributed customers
Operations Staff	no_ops_session / no_ops_csrf	/staff/*, /operations/*, /admin/* (per permission)	Named permission list only
Admin	no_ops_session / no_ops_csrf	everything Operations Staff can, implicitly	Every permission granted implicitly by role

3. Stage-by-stage summary (10-13)

Stage 10 — Foundation & public website

The design system, global header/footer, homepage, service pages, destination pages, and offer pages — Arabic-first, fully responsive (390-1440px), WCAG-conscious from the start.

- Design tokens, typography, spacing and grid system, componentised UI
- 13 services, 11 destinations, 3 offers, all real content and photography

Verification: tests/home, services, detail, destinations, offers, ghx, ghm, gfx, final — all green

Stage 11 — Booking journey

The flight search-to-confirmation journey: location combobox, results, offer details, traveller forms, extras, review, payment, and confirmation — plus the request-only path for services without live inventory.

- A documented, swappable supplier adapter contract (search/offer/quote/book/extras)
- A development flight adapter, clearly labelled, never presented as real inventory
- Request-only booking for hotels/visa/packages/Umrah/medical/transport

Verification: tests/booking, journey — 236 and 512 checks, all green

Stage 12 / 12.1 / 12.2 — Customer accounts & backend

Full customer authentication and account area (trips, bookings, travellers, documents, payments, notifications, settings), a documented API contract, and a real deployable Node.js/SQLite backend implementing that contract end to end.

- HttpOnly + Secure + SameSite cookies, CSRF, exact-origin CORS, rate limiting, logout
- Private document storage with signed, expiring, revocable URLs — never a permanent public link
- A configuration layer that refuses to start in production with an unsafe setting

Verification: tests/account, integration, backend — 782, 518, and (post-Stage 15B) 201 checks, all green

Stage 13 — Supervisor system

A separate credential/session store for supervisors, public profile URLs, first-touch customer attribution (never silently overwritten), a scoped portal (customers/bookings/leads/revenue/performance), and a disabled-by-default admin reassignment endpoint that preserves full attribution history.

- Attribution survives navigation, authentication, booking, payment and confirmation
- Commission structure present but deliberately unconfigured — no invented percentage or amount

Verification: tests/supervisor, supervisor-portal — 182 and 505 checks, all green

4. Stage 14 — Admin Dashboard

The management/oversight layer above Stage 15's operational engine — never a rebuild or a duplicate of it. Adds cross-customer and cross-supervisor visibility, staff account management, and descriptive reporting, reusing every domain function Stages 12-13-15 already built.

Modules added

- Customers — admin-wide list/detail, unified view (bookings, documents, payments, attribution history)
- Supervisors — admin-wide list/create/detail/update, composing the same scoped read models the supervisor portal itself uses
- Leads / Attribution — admin-wide read of the same tables the supervisor portal scopes to one supervisor
- Payments & Documents — read-only admin-wide browse, never a storage key or a settlement calculation
- Reports — five descriptive reports (bookings, operations, suppliers, documents, notifications), plain counts and distributions, nothing ranked or scored
- Staff & Permissions — account provisioning via the existing password-reset flow (an admin never handles a password), activate/deactivate, explicit permission grants
- Global search — bounded, permission-scoped, across customers/bookings/supervisors/suppliers/tasks/escalations

Permissions added

Permission	Gates
customer.view	Overview, Customers, search's customer/booking categories
supervisor.view / supervisor.manage	Supervisors list/detail — and create/update/reassign
payment.view	Admin-wide payments
document.view	Admin-wide document browse
attribution.view	Leads, attribution history
staff.manage	Staff account provisioning and permission grants

Verification: tests/backend.mjs (+20 checks), tests/ops-portal.mjs (+55 checks) — all green. Full regression run before commit 6540ee9.

5. Stage 15 — Operations Control Layer

The operational engine Stage 14 sits above: a backend-authoritative booking lifecycle state machine (separate from the customer-facing status), tasks, escalations, a service operational catalogue with per-service workflow and document requirements, document review, a supplier directory, customer/internal note separation, notification templates, and a full audit trail.

- Every operational status transition is validated server-side and recorded in `booking_status_history`
- Payment-gated transitions (e.g. entering 'processing') are rejected (409) unless `payment_status='paid'`
- Internal notes are never returned by any customer-facing route
- 13 services registered; 4 (flights, visa, hotels, medical) have a configured workflow at this stage

Verification: `tests/backend.mjs` Stage 15 section, `tests/ops-portal.mjs` — all green.

6. Stage 15A — Business Rules Register

An audit of the codebase found that every business rule Stage 15's own status report called "unresolved" already had an honest answer in `business_config` (seeded since Stage 10/13/15) — most as an explicit `PENDING` row with a null value. What was missing was a formal, versioned register with the schema a real business-rules system needs. Stage 15A built exactly that, without deciding a single business value.

The ten registered rules

Rule	Category	Status	Value
Booking lifecycle graph	booking_lifecycle	DRAFT	Stage 15's worked-example graph
Payment gates	payment	DRAFT	the technical <code>PAYMENT_GATED</code> list
Supervisor attribution model	attribution	DRAFT	first-touch, in effect since Stage 10
Task priority levels	task_priority	DRAFT	[low, normal, high, urgent]
Commission model	commission	PENDING	null
Refund policy	refund	PENDING	null
Cancellation policy	cancellation	PENDING	null
SLA targets	sla	PENDING	null
Staff provisioning	staff	PENDING	mechanism built, no real hire
Admin Dashboard scope	admin	ACTIVE	the delivered module list

What the register provides

- Full schema — `rule_id`, `category`, `name`, `description`, `current_value`, `allowed_values`, `status`, `source`, `effective_from/to`, `updated_by/at`, `notes`
- Versioning — every change archives the prior version to `business_config_history` before overwriting; nothing is ever silently lost
- A Pending Decision Center — every `PENDING/DRAFT` rule plus unconfigured services and unconnected suppliers, computed, not duplicated into a second table
- A Final Business Rule Matrix — register rules plus computed coverage counts (services, suppliers)
- Two new permissions — `rules.view` / `rules.manage`, enforced identically to every other Stage 14 permission

Verification: `tests/backend.mjs` (+20 checks), `tests/ops-portal.mjs` (+2 checks) — all green. Commit `fbe0284`.

7. Stage 15B — Apply & Verify Business Rules

Auditing the register against the live domain code found that three functions (`lifecycleConfig()`, `taskPriorityLevels()`, `commissionModel()`) already read their value live from `business_config` on every call — but each returned a status string embedded in that same JSON value, a second, independently stale copy of what the register's own status column said. Activating a rule from the Admin Dashboard changed the column but not the string any live consumer actually read.

What changed

- All three functions now derive their status from the register's own status column — one source of truth
- Payment gates converted from a hardcoded constant to a live register read (identical default values, now genuinely admin-editable)
- The Business Rules manage form gained a raw JSON value editor (`rules.manage` only, client-side validated)

Protection invariants proven (not assumed)

- No code path anywhere computes a commission amount, a refund amount, a cancellation fee, or an SLA breach — confirmed by direct repository search, then locked in as a regression test
- A customer reading their own booking never sees an internal operations field (`opsStatus`, `assignedOperator`)
- A customer or supervisor session can never reach a staff-only mutation — three new direct-API checks

Verification: `tests/backend.mjs` (+16 checks, total 201), `tests/ops-portal.mjs` (+2 checks, total 900) — all green. Full regression (21 suites) green. Commit `baaf56e`.

8. Stage 16 — Production Integration Audit

Overall status: PARTIALLY COMPLETE — not a shortfall in effort, but the correct, honest outcome this stage's own rules require.

No real payment provider, flight or hotel supplier, SMS/email/WhatsApp provider, hosted identity service, or production hosting account exists anywhere in this project. Stage 16's own brief repeatedly and explicitly forbids inventing any of these or marking them CONNECTED without genuine evidence. Following that instruction to the letter means this stage delivers an audit and a runbook, not live connections.

What the audit verified

- Environment separation (development/staging/production) is real and tested: production refuses test controls, a missing or weak signing secret, a non-https origin, an insecure cookie, or an unimplemented storage/mailer value — never silently falls back to a mock
- The deployment sequence (tools/deploy.mjs) already matches this stage's own required order: config check → migrations → smoke test → real-backend acceptance suite → only then mark verified and publish
- A documented, swappable adapter contract already exists for the flight supplier (assets/js/booking/adapters/index.js) and the payment provider (assets/js/booking/payment.js) — a real provider is one registered adapter away
- Services without a live supplier (hotels, packages, visa, Umrah, medical, transport) correctly run through a request-only path — no fabricated search result exists anywhere

Genuine gap found and documented (not fixed speculatively)

backend/routes.mjs's booking-finalize route (me.claim) sets payment_status='paid' from a value the **browser** reports in its request body — safe today only because the only payment provider is the labelled development stand-in. This is exactly the risk Stage 16 warns against: *"never trust a browser success page as proof of payment."* The documented fix — a server-side payment intent plus a signature-verified webhook, never a client-reported status — is recorded in docs/PRODUCTION-INTEGRATION.md §3, to be built alongside whichever real provider is chosen (building it speculatively now risks the wrong shape).

Full detail: docs/PRODUCTION-INTEGRATION.md. Commit 45fe35c.

9. Production Integration Matrix

System	Provider	Status	Verified	Notes
Authentication	backend-managed	NOT_CONNECTED	N/A	mechanism implemented and tested; no deployed instance
Customer Backend	backend/ (this repo)	NOT_CONNECTED	N/A	implemented, not deployed anywhere
Supervisor System	backend-managed	NOT_CONNECTED	N/A	same backend, same deployment gap
Flight Supplier	none chosen	NOT_CONNECTED	N/A	dev adapter only, clearly labelled
Hotel Supplier	none chosen	NOT_CONFIGURED	N/A	correctly routed to request-only, no search exists
Payment Provider	none chosen	NOT_CONNECTED	N/A	dev provider only; webhook fix required first
Document Storage	local disk	CONNECTED (local)	YES	private, signed URLs, tested; S3 seam NOT_CONFIGURED
Email	none chosen	NOT_CONNECTED	N/A	outbox records intent only
SMS	none chosen	NOT_CONNECTED	N/A	same seam as email
WhatsApp	none chosen	NOT_CONNECTED	N/A	same seam
Database	local SQLite file	CONNECTED (local)	YES	migrations tested; no production host
Staff Identity	backend-managed	NOT_CONNECTED	N/A	mechanism real; only fixture accounts exist
Legal Content	fixture text	NOT_CONFIGURED	N/A	placeholder terms/privacy, explicitly labelled

No row claims CONNECTED or VERIFIED without the evidence stated in its Notes column.

10. Test results summary

The most recent full regression run (21 suites plus the internationalisation and accessibility audits), executed before Stage 15B/16 were committed:

Suite	Result
final / ghx / ghm / gfx	96 / 16 / 42 / 62 checks — all passed
home / services / detail / destinations / offers	170 / 146 / 470 / 165 / 170 checks — all passed
booking / journey	236 / 512 checks — all passed, 0 console/network problems
account / integration	782 / 518 checks — all passed, 0 console/network problems
backend	201 checks — all passed
supervisor / supervisor-portal	182 / 505 checks — all passed
ops-portal	900 checks — all passed, 0 console/network problems
rm (motion/keyboard) / links	27 checks / 0 problems
i18n audit	0 untranslated strings
a11y audit	0 unique findings

11. Pending business decisions

None of the following were resolved by this project, and none should be silently defaulted:

- Final booking lifecycle graph — the current graph is a working technical example, not confirmed policy
- SLA target durations, warning and escalation thresholds
- Task priority ordering and its severity meaning
- Required documents and workflows for the 9 services still unconfigured
- Real supplier integrations (flight, hotel, and any other live-inventory service)
- The notification delivery pipeline (a real email/SMS/WhatsApp provider)
- Commission, refund, and cancellation policy values
- Real staff provisioning — who the actual Admin/Operations Staff are
- Choice of hosting, payment provider, and their commercial terms

12. Known limitations

- The Business Rules value editor is a raw JSON textarea, not a schema-aware form per rule type — acceptable at the current scale (10 rules), worth revisiting if the register grows much larger
- Global admin search is a bounded LIKE match, not a ranked or fuzzy search
- The booking-finalize route trusts a client-reported payment status — safe only until a real payment provider is connected (see §8); the fix is designed and documented, not yet built
- No external monitoring service (error tracking, uptime) is connected — none has been chosen
- Backup and recovery cannot be verified — no hosting has been chosen to back up

13. Recommended next steps

In the order that unblocks the most work:

- **1. Choose and provision hosting** for the backend (a host for backend/Dockerfile with a persistent volume and TLS) and the static frontend — this alone lets Stage 16's smoke tests actually run
- **2. Approve the pending business rules** that block real operations first: the booking lifecycle graph and payment gates (both already have a working technical default awaiting confirmation)
- **3. Choose a payment provider** and build the webhook-verified payment flow documented in docs/PRODUCTION-INTEGRATION.md §3, closing the client-trust gap before any real transaction is possible
- **4. Choose a flight (and, if desired, hotel) supplier** and register a real adapter alongside the existing contract — no other booking-flow code needs to change
- **5. Supply approved legal content** (Terms of Service, Privacy Policy, Arabic and English, versioned) and provision the real Admin/Operations Staff who will use the dashboard
- **6. Choose a notification provider** (email at minimum) so the outbox can move from queued to genuinely delivered

This report was generated from the repository's own committed history and automated test results as of commit 45fe35c on branch claude/awesome-feynman-zyfjar. No figure in it was estimated or assumed.